

CyberLab Deployment Runbook

Oct 3, 2026 · maintained by @Saaaid

About this runbook

This runbook takes you from nothing to a working CyberLab two ways: **locally** on your own computer with Oracle VirtualBox, or **remotely** on an **AWS Academy Learner Lab** EC2 instance. Both run the identical segmented range, so any lab works unchanged on either.

It is written for two audiences:

- **Instructors** deploy and manage the range — follow **Part A** (VirtualBox) or **Part B** (AWS), then share the address with your class.
- **Students** connect to a range the instructor already started and work the labs — go straight to **Part C**.

Conventions used throughout: text in `monospace` is typed exactly; replace anything in `<angle brackets>` with your own value; a line beginning `sudo` is run on the lab host, not inside a container.

Index

#	Section	For	What it covers
1	About this runbook	Everyone	Who does what, conventions
2	Index	Everyone	This table
3	What you're deploying	Everyone	The segmented range, zones, hosts
4	Prerequisites	Instructor	Hardware, software, accounts
5	Part A — Local with VirtualBox	Instructor	Build the VM and start the range on your computer
6	Part B — Remote on AWS Learner Lab	Instructor	Launch an EC2 instance and start the range in the cloud
7	Part C — Student access & doing a lab	Student	Connect, verify, pivot, submit

#	Section	For	What it covers
8	Managing the range	Instructor	The <code>cyberlab</code> command
9	Optional: CAI agent	Instructor	Add the AI pentest agent
10	Troubleshooting	Everyone	Common problems and fixes
11	Quick reference	Everyone	One-screen cheat sheet
12	Security & acceptable use	Everyone	The rules that keep this legal and safe

What you're deploying

CyberLab is **not** one flat network. It is a company-style network split into three zones behind a firewall router, all running as Docker containers on one Linux host:

```

INTERNET 10.10.0.0/24  attacker 10.10.0.10  ---  router 10.10.0.254
                                     |  (firewall:
80/3000/53/icmp)
DMZ       10.20.0.0/24  juiceshop .11 (web 3000) | dvwa .12 (web 80) | dns
.53
                                     |  (pivot through dvwa)
LAN       10.30.0.0/24  metasploitable .20  (reachable ONLY via the dvwa
pivot)

```

The **attacker** workstation (Kali + an XFCE desktop over RDP) is where all work happens. The **firewall router** lets the attacker reach the DMZ only on web ports (80, 3000), DNS (53) and ping. The **LAN** has no route in — students reach `metasploitable` by pivoting through the dual-homed `dvwa`. A **Suricata** IDS sensor and a **Portainer** management GUI round it out.

Zone	Host	IP	Role
Internet	<code>attacker</code>	10.10.0.10	Kali tools + XFCE/RDP desktop (every lab)
Internet/DMZ	<code>router</code>	10.10.0.254 / 10.20.0.254	Firewall / NAT between zones
DMZ	<code>juiceshop</code>	10.20.0.11	OWASP Juice Shop (web, port 3000)

Zone	Host	IP	Role
DMZ	<code>dvwa</code>	10.20.0.12 / 10.30.0.12	DVWA (web, port 80) — dual-homed LAN pivot
DMZ	<code>dns</code>	10.20.0.53	BIND, authoritative for <code>cipher.lab</code>
LAN	<code>metasploitable</code>	10.30.0.20	Vulnerable Linux (via pivot)
host net	<code>suricata</code>	br-inet + br-lan	IDS sensor
host-only	<code>portainer</code>	127.0.0.1:9443	Container management GUI

The labs use a fictional engagement — **Summit Cyber Group** (the consultancy you work for) assessing **Cipher Logistics** (the client) — purely as narrative framing.

Prerequisites (instructor)

Need	Local (VirtualBox)	Remote (AWS Learner Lab)
Host	A 64-bit x86 computer with virtualization enabled in BIOS	An AWS Academy Learner Lab course seat
CPU / RAM	4+ vCPU, 8 GB min / 16 GB recommended free for the VM	<code>t3.large</code> (8 GB) min, <code>t3.xlarge</code> (16 GB) recommended
Disk	~60 GB free	50 GB gp3 EBS volume
Software	Oracle VirtualBox; an Ubuntu Server 24.04 (or Kali) ISO	A browser; an SSH client; an RDP client
Network	Internet for the first build (~3 GB)	Internet (default)
Files	<code>bootstrap-local.sh</code> from the repo	<code>bootstrap-vps.sh</code> from the repo

Important: the target images (DVWA, Metasploitable) are **x86-only** — do not use an Arm/Apple-silicon machine or an Arm cloud instance. Linux host networking is required (for the Suricata sensor), so this runs on a real Linux host/VM, not Docker Desktop on macOS/Windows.

Get the project first:

```
git clone https://github.com/ielhadad/CyberLab.git
```

The bootstrap files

The repo ships three ready-to-run scripts. Each is **self-contained** — it writes everything it needs under `/opt/cyberlab` and installs the `cyberlab` management command — so you do not even need to clone the repo on the lab host to use one.

Script	Purpose	Used in
<code>bootstrap-local.sh</code>	Installs Docker, builds and starts the whole range, and installs the <code>cyberlab</code> helper — for a host you manage directly.	Part A
<code>bootstrap-vps.sh</code>	Everything <code>bootstrap-local.sh</code> does plus cloud hardening: a swap file, an SSH-only firewall (ufw), fail2ban, and automatic security updates.	Part B
<code>hcloud-launch.sh</code>	Convenience launcher that creates a Hetzner Cloud server and runs <code>bootstrap-vps.sh</code> on it automatically. Not needed for AWS.	Hetzner only

For this runbook you use `bootstrap-local.sh` in Part A and `bootstrap-vps.sh` in Part B.

Part A — Local deployment with VirtualBox (instructor)

Use this to run the range on your own computer. Budget 30–45 minutes, most of it the one-time image build.

1. **Install VirtualBox.** Download and install Oracle VirtualBox from [virtualbox.org](https://www.virtualbox.org) for your OS.
2. **Download the OS ISO.** Get the **Ubuntu Server 24.04 LTS** ISO (Kali also works).
3. **Create the VM.** In VirtualBox: *New* → name it `cyberlab-host`, Type *Linux*, Version *Ubuntu (64-bit)*. Give it **4+ CPUs**, **8192 MB+ RAM** (16384 if you can), and a **60 GB** virtual disk.
4. **Set networking.** In the VM's *Settings* → *Network*, use a **Bridged** or **Host-Only** adapter so you can reach the VM, and note the VM's IP after install (`ip a`).
5. **Install Ubuntu.** Attach the ISO, boot the VM, and run the installer. Create your login user, and enable **OpenSSH server** when the installer offers it.

6. **Snapshot the clean install.** Power off and take a VirtualBox *Snapshot* ("clean install") so you can roll back later in seconds.

7. **Copy the bootstrap in.** Boot the VM, then from your computer copy the script over:

```
scp bootstrap-local.sh <user>@<vm-ip>:~
```

8. **Run it.** SSH into the VM and run the bootstrap (it installs Docker, builds the attacker desktop image, and starts everything):

```
ssh <user>@<vm-ip>
sudo bash bootstrap-local.sh
```

9. **Wait for the build.** The first run pulls ~3 GB and builds the Kali + XFCE/XRDP image — several minutes. Watch progress with `tail -f /var/log/cyberlab-bootstrap.log`.

10. **Verify.** When it finishes, run `cyberlab verify` — it should end with **"All checks passed."** If a check or two fail on first boot, wait a minute (Suricata loads rules) and run it again.

11. **Share access with students.** Give students the VM's IP and the connection steps in Part C. For GUI labs they tunnel RDP; for CLI labs they use `docker exec`.

To reset between sessions: `cd range && docker compose down -v && docker compose up -d`, or use the snapshot.

Part B — Remote deployment on AWS Academy Learner Lab (instructor)

Use this to run the range in the cloud so students connect from anywhere. The Learner Lab is a time-boxed, budget-capped sandbox, so build early and snapshot an image.

1. **Start the Learner Lab.** In AWS Academy, open your **Learner Lab**, click **Start Lab**, wait for the dot to turn green, then click **AWS** to open the console. Under **AWS Details**, download the lab's **SSH key** (`labsuser.pem`) — you cannot make your own key pair here.

2. **Launch an instance.** EC2 → *Launch instances*. Name it `cyberlab-host`; AMI **Ubuntu Server 24.04 LTS (x86)**; type `t3.xlarge` (or `t3.large` minimum — avoid larger types the sandbox may block).

3. **Key pair.** Choose the Learner Lab's existing key pair (`vockey` / `labsuser.pem`).

4. **Security group — SSH only.** Create a group with a **single inbound rule**: SSH (22) from **My IP**. Remove every other inbound rule; do **not** open 3389, 80, 3000 or 9443.

5. **Storage.** Set the root volume to **50 GiB, gp3**.
6. **Launch**, then note the instance's **Public IPv4**.
7. **Connect over SSH.** On your computer:

```
chmod 400 labsuser.pem
ssh -i labsuser.pem ubuntu@<PUBLIC_IP>
```

8. **Run the bootstrap.** Copy `bootstrap-vps.sh` up (`scp -i labsuser.pem bootstrap-vps.sh ubuntu@<PUBLIC_IP>:~`) and run it — it also adds swap, an SSH-only firewall, fail2ban and auto-updates:

```
sudo bash bootstrap-vps.sh
```

(Or paste its contents into the instance's **User data** at launch to run it automatically.)

9. **Wait and verify.** `tail -f /var/log/cyberlab-bootstrap.log` ; when done, `cyberlab verify` should report **"All checks passed."**
10. **Snapshot an AMI.** EC2 → select the instance → *Actions* → *Image and templates* → *Create image*. Launch future hosts from that AMI so they skip the slow build — important, because the Learner Lab resets.
11. **Share access.** Students reach the GUI desktop over an **SSH tunnel** (Part C); nothing is exposed publicly.

Learner Lab specifics to plan around:

- The sandbox **stops your instances when the session ends** and has a fixed budget — build and snapshot early, and expect to start/stop around class windows.
- Anything not on the EBS volume (or captured in an AMI) is lost on reset.
- Keep the security group to **SSH-from-your-IP only** — never open RDP to the internet.
- **Stop** the instance when class isn't running to conserve the budget.

Part C — Student access & doing a lab

Your instructor runs the range and gives you its address. Everything you do starts from the **attacker** workstation.

1. Connect to the attacker

- **GUI labs (Zenmap, Burp, Wireshark, Ettercap):** open an RDP desktop.

- *Local VirtualBox host*: point a Remote Desktop client at **127.0.0.1:3389** if you're on the host, or at the VM's IP as your instructor directs.
- *AWS / remote host*: first open an SSH tunnel on your laptop, then RDP to **127.0.0.1:3389**:

```
ssh -i labsuser.pem -L 3389:127.0.0.1:3389 ubuntu@<PUBLIC_IP>
```

- Log in **root / cyberrange**.
- **CLI labs**: on the lab host, `docker exec -it attacker bash`.

2. Verify you can reach the DMZ

```
whoami && hostname          # root @ attacker
nmap -sn 10.20.0.0/24        # the DMZ hosts answer
curl -s http://10.20.0.11:3000 >/dev/null && echo OK  # juiceshop reachable
```

3. Know your targets

Zone	Name	IP	What it is
DMZ	juiceshop	10.20.0.11	web app (port 3000)
DMZ	dvwa	10.20.0.12	web app (port 80) — also the LAN pivot
DMZ	dns	10.20.0.53	DNS for <code>cipher.lab</code>
LAN	metasploitable	10.30.0.20	vulnerable Linux (via pivot)

4. Reaching the LAN (pivot)

Some labs target `metasploitable` on the internal LAN. The firewall won't route you there directly — you go **through** `dvwa`, which is dual-homed (10.20.0.12 in the DMZ, 10.30.0.12 on the LAN) and deliberately vulnerable (its Command Injection page gives you execution on a host that sits on the LAN). Each lab's *Reaching the target* box says whether you need the pivot or can use a DMZ/offline fallback.

5. Do the lab and submit

Each lab offers three tracks — **Guided** (commands + expected output), **Unguided** (objectives + hints), and **Capture-the-Flag** (flags, no instructions). Work the track your instructor set, then submit the **Deliverable**: screenshots showing your full window (uncropped), the terminal-activity-log step, and a short written report of what you did and why it worked.

Managing the range (the `cyberlab` command)

The bootstrap installs a `cyberlab` helper on the host for day-to-day management:

Command	What it does
<code>cyberlab up</code>	Build + start the range, then print access info
<code>cyberlab verify</code>	Self-test: segmentation, DNS, pivot, IDS and service checks
<code>cyberlab status</code>	Show container status
<code>cyberlab alerts</code>	Tail the Suricata IDS alerts
<code>cyberlab scan <url></code>	Run a nuclei scan from the attacker (e.g. <code>cyberlab scan http://10.20.0.12</code>)
<code>cyberlab cai</code>	Enable the optional CAI agent (needs an API key)
<code>cyberlab reset</code>	Recreate containers; add <code>--full</code> to also wipe volumes
<code>cyberlab down</code>	Stop everything
<code>cyberlab destroy</code>	Remove containers, volumes and images

Optional: the CAI agent

CAI (Cybersecurity AI) is an optional AI-assisted pentest agent that drives tools from a plain-language goal. It is **off by default** because it calls an external LLM API and needs a key.

To enable it on the host:

1. Run `cyberlab cai` once — it creates `/opt/cyberlab/cai/cai.env`.
2. Edit that file and add an `OPENAI_API_KEY` or `ANTHROPIC_API_KEY` (and a matching `CAI_MODEL`).
3. Run `cyberlab cai` again — it builds and starts the agent at `10.10.0.60`.
4. Use it via `docker exec -it cai cai`, or the browser terminal at `http://127.0.0.1:7682` (tunnel port 7682 on a remote host).

Keep CAI scoped to the range: the DMZ (`10.20.0.0/24`) and, via the dvwa pivot, `10.30.0.20` . Set a spend limit on your API account before classroom use.

Troubleshooting

Symptom	Likely cause → fix
First build is very slow	Normal — it builds the Kali desktop image and pulls ~3 GB. Watch <code>tail -f /var/log/cyberlab-bootstrap.log</code> .
Can't RDP to <code>127.0.0.1:3389</code>	On a remote host you must open the SSH tunnel first (<code>ssh -L 3389:127.0.0.1:3389 ...</code>). Locally, check <code>docker ps</code> shows <code>attacker</code> up.
<code>nmap -sn 10.20.0.0/24</code> finds nothing	Missing route on the attacker: <code>ip route replace 10.20.0.0/24 via 10.10.0.254</code> .
A DMZ host shows few open ports	Expected — the firewall forwards only 80/3000/53/icmp into the DMZ.
Can't reach <code>10.30.0.20</code> directly	Expected — the LAN has no route in. Use the dvwa pivot.
<code>cyberlab verify</code> fails a check	Give it a minute on first boot (Suricata loads rules), then run it again.
AWS instance unreachable after a day	The Learner Lab stopped it at session end. Start the lab again and the instance.
<code>command not found</code> in the attacker	Install it: <code>apt-get update && apt-get install -y <tool></code> .
Out of disk during build	Use a 50–60 GB disk; remove old images with <code>docker system prune -a</code> .

Quick reference

```

Deploy (local) : sudo bash bootstrap-local.sh
Deploy (cloud) : sudo bash bootstrap-vps.sh
Manage        : cyberlab up | verify | status | alerts | scan <url> | reset
               | down

Connect GUI    : (remote) ssh -L 3389:127.0.0.1:3389 <user>@<host>
               then RDP to 127.0.0.1:3389 (root / cyberrange)
Connect CLI    : docker exec -it attacker bash

DMZ web        : http://10.20.0.11:3000 (juiceshop) http://10.20.0.12
               (dvwa)
DMZ DNS        : dig @10.20.0.53 <name>.cipher.lab
LAN target     : 10.30.0.20 (metasploitable) – via the dvwa pivot
Portainer      : https://127.0.0.1:9443 (tunnel 9443 on a remote host)

Reset range    : cd range && docker compose down -v && docker compose up -d
Log            : tail -f /var/log/cyberlab-bootstrap.log

```

Security & acceptable use

CyberLab installs real offensive security tooling. It is safe only because it is contained — keep it that way.

- **Only attack hosts inside the range.** Never point these tools at a real website, a classmate's machine, the campus network, or anything outside the lab.
- **Nothing is exposed publicly.** RDP, Portainer and SOCKS are bound to `127.0.0.1`; reach them over an SSH tunnel. On a cloud host, open only SSH (22) to your own IP — never open 3389 to the internet.
- **Credentials are lab-only.** `root / cyberrange` and the app logins are for this disposable range; never reuse them anywhere real.
- **The instructor answer key is confidential** and is shared only with verified instructors — it is not part of the student clone.
- Tear down cloud instances when done, and treat the whole range as disposable.

See `DISCLAIMER.md` and `ACCEPTABLE_USE.md` in the repository for the full terms.